

C:\Nia\Nia.psd1

11/30/25 9:10 AM

```
C:\Nia\  
├─ Nia.psm1  
├─ Nia.psd1  
└─ core\  
    ├── llm.ps1  
    ├── vault.ps1  
    └── memory.ps1  
        └─ util.ps1  
C:\Nia\Nia.psd1  
@{  
    RootModule      =  
'Nia.psm1'  
    ModuleVersion   = '1.0.0'  
    GUID            = 'c28be6ca-  
c042-4df5-b77e-  
d377329afa88'  
    Author          = 'Jason  
Lesane'  
    CompanyName     = 'House  
of Jazzu'  
    Copyright       = "  
    PowerShellVersion = '5.1'  
    Description      = 'Nia —  
Agentic AI Assistant & CEO of  
House of Jazzu'  
    FunctionsToExport = @(  
        'Nia', 'Get-NiaStatus',  
'Send-NiaCommand',  
        'Nia-Advise', 'Nia-  
RunLLM',  
        'Nia-VaultStore', 'Nia-  
VaultGet'  
    )  
    FileList = @(  
        'Nia.psm1',  
        'core\llm.ps1',  
        'core\vault.ps1',  
        'core\memory.ps1',  
        'core\util.ps1'  
    )  
}  
C:\Nia\Nia.psm1  
# Nia PowerShell Module —  
Clean Build  
# Loads submodules and  
exports public commands  
  
$moduleRoot = $PSScriptRoot  
  
# Load submodules  
."$moduleRoot\core\util.ps1"  
."$moduleRoot\core  
\memory.ps1"  
."$moduleRoot\core  
\vault.ps1"  
."$moduleRoot\core\llm.ps1"  
  
function Get-NiaStatus {  
    [PSCustomObject]@{  
        Name      = "Nia"  
        Status    = "Online"  
        Version   = "1.0"  
        Memory    = "Quantum  
Encrypted"  
        Vault     = "Active"  
        LLM       = "Ready"  
    }  
}  
  
function Send-NiaCommand {  
  
    param([Parameter(Mandatory  
)] [string] $Message)  
    Write-Host "Nia  
acknowledges: $Message" -  
ForegroundColor Cyan  
}  
  
# Simple chat wrapper  
function Nia {  
  
    param([Parameter(Mandatory  
)] [string] $Input)  
    Nia-RunLLM -Prompt $Input  
}  
  
Export-ModuleMember -  
Function * -Alias *  
function Write-Nia {  
    param([string] $Text)  
    Write-Host "[Nia] $Text" -  
ForegroundColor Magenta  
}  
# Simple persistent JSON  
memory store  
  
$MemoryFile =  
"$PSScriptRoot\..  
\nia_memory.json"  
  
if (!(Test-Path $MemoryFile)) {  
    '{}' | Out-File $MemoryFile  
}  
  
function Nia-Remember {  
  
    param([Parameter(Mandatory  
)] [string] $Key,  
        [Parameter(Mandatory)]  
[string] $Value)  
  
    $mem = Get-Content  
$MemoryFile | ConvertFrom-  
Json  
    $mem | Add-Member -  
NotePropertyName $Key -  
NotePropertyValue $Value -  
Force  
    $mem | ConvertTo-Json |  
Out-File $MemoryFile  
}  
  
function Nia-Recall {  
  
    param([Parameter(Mandatory  
)] [string] $Key)  
    (Get-Content $MemoryFile  
| ConvertFrom-Json).$Key  
}  
# Simple persistent JSON  
memory store  
  
$MemoryFile =  
"$PSScriptRoot\..  
\nia_memory.json"  
  
if (!(Test-Path $MemoryFile)) {  
    '{}' | Out-File $MemoryFile  
}  
  
function Nia-Remember {  
  
    param([Parameter(Mandatory  
)] [string] $Key,  
        [Parameter(Mandatory)]  
[string] $Value)  
  
    $mem = Get-Content  
$MemoryFile | ConvertFrom-  
Json  
    $mem | Add-Member -  
NotePropertyName $Key -  
NotePropertyValue $Value -  
Force  
    $mem | ConvertTo-Json |  
Out-File $MemoryFile  
}  
  
function Nia-Recall {  
  
    param([Parameter(Mandatory  
)] [string] $Key)  
    (Get-Content $MemoryFile  
| ConvertFrom-Json).$Key  
}  
# AES-256 encrypted vault file  
  
$VaultPath = "$PSScriptRoot  
\..\nia_vault.dat"  
  
function Nia-VaultStore {  
    param(  
        [Parameter(Mandatory)]  
[string] $Key,  
        [Parameter(Mandatory)]  
[string] $Value  
    )  
    $data = @{  
        key   = $Key  
        value = $Value  
        time  = (Get-Date)  
    } | ConvertTo-Json  
  
    $secure = ConvertTo-  
SecureString $data -  
AsPlainText -Force  
    $secure | ConvertFrom-  
SecureString | Out-File  
$VaultPath  
    Write-Nia "Value stored  
securely."  
}  
  
function Nia-VaultGet {  
  
    param([Parameter(Mandatory  
)] [string] $Key)  
  
    if (!(Test-Path $VaultPath)) {  
        Write-Nia "Vault empty."  
        return  
    }  
  
    $raw = Get-Content  
$VaultPath  
    $plain = (ConvertTo-  
SecureString $raw |  
ConvertFrom-SecureString -  
AsPlainText)  
    ($plain | ConvertFrom-  
Json).value  
}  
function Nia-RunLLM {  
  
    param([Parameter(Mandatory  
)] [string] $Prompt)  
  
    # In production: call X.ai or  
your Grok API endpoint  
    return "Nia (LLM): I hear  
you. Processing: $Prompt"  
}  
  
function Nia-Advise {  
    param([string] $Topic)  
    Nia-RunLLM -Prompt  
"Advise me about: $Topic"  
}  
Copy-Item "C:\Nia" "$HOME  
\Documents\PowerShell  
\Modules" -Recurse -Force  
Copy-Item "C:\Nia" "$HOME  
\Documents  
\WindowsPowerShell  
\Modules" -Recurse -Force  
Import-Module Nia -Force  
Get-NiaStatus  
Name      Status Version  
Memory    Vault   LLM  
-----  
Nia      Online 1.0   Quantum  
Encrypted Active  Ready  
Add-Content $PROFILE  
'Import-Module Nia -Force'  
. $PROFILE
```

